0001. Two Sum

: Easy | Category: Arrays & Hashing | LeetCode: https://leetcode.com/problems/t

Problem

- Given nums and target, return the indices of the two numbers that add up to target.
- Exactly one solution exists, and you may not use the same element twice.
- Example: nums = [2,7,11,15], target = 9 → [0,1].

Intuition

- For each x, the partner we need is target - x (its complement).
- If we remember every value we have already passed (value → index), we can check for the complement in O(1).

Brute Force

- Try every pair i < j and test nums[i] + nums[j] == target: O(n^2) time, O(1) space.

Optimal Approach

- Keep a HashMap from value to index.
- For index i with value x: if map contains target - x, return [map.get(target - x), i].
- Otherwise store x → i and continue.
- Checking before inserting guarantees we never pair an element with itself.

Worked Example

```
nums = [3, 2, 4], target = 6.

i=0 x=3 need 3  map={}          -> store 3:0
i=1 x=2 need 4  map={3:0}       -> store 2:1
i=2 x=4 need 2  map={3:0,2:1}   -> found at 1 -> return [1,2]
```

Java Solution

```java
public class TwoSum {
    public int[] twoSum(int[] nums, int target) {
        Map<Integer, Integer> index = new HashMap<>();
        for (int i = 0; i < nums.length; i++) {
            Integer j = index.get(target - nums[i]);
            if (j != null) return new int[]{j, i};
            index.put(nums[i], i);
        }
        return new int[0];
    }

    public int[] twoSumBrute(int[] nums, int target) {
        for (int i = 0; i < nums.length; i++)
            for (int j = i + 1; j < nums.length; j++)
                if (nums[i] + nums[j] == target) return new int[]{i, j};
        return new int[0];
    }
}
```

Complexity

- Time: O(n), one pass.
- Space: O(n) for the map.

Pitfalls

- Inserting before checking breaks cases like [3,3] target 6 only if you overwrite; checking first is the clean fix.
- Return indices, not values. Sorting destroys the original indices.